ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ

BÁO CÁO LỜI GIẢI BÀI TẬP VỀ NHÀ

MÔN HỌC: PHÂN TÍCH DỮ LIỆU VÀ XÂY DỰNG MÔ HÌNH DATA-DRIVEN – MACHINE LEARNING ỨNG DỤNG TRONG KỸ THUẬT

Giảng viên hướng dẫn: TS. Phùng Thanh Huy

Sinh viên thực hiện: Hồ Quốc Khang

MSSV: 2511491

TP. HỒ CHÍ MINH – 2026

Mục lục

Cụm 1. Tiền xử lý dữ liệu và khảo sát chuỗi thời gian cảm biến **1**
Câu 1. Thống kê bản ghi trùng, giá trị thiếu và điểm nghi nhiễu từ bộ dữ liệu raw 1
Câu 2. Cơ chế nội suy và loại bỏ giá trị thiếu cho cảm biến nhiệt độ và dòng điện 5

Cụm 2. Mô hình hồi quy nhiệt độ và đánh giá sai số dự báo **7**
Câu 3. So sánh MAE, RMSE, R^2 của các mô hình hồi quy và cơ chế sai số 7

Cụm 3. Mô hình phân loại cảnh báo lỗi hiếm và kiểm soát độ tin cậy **10**
Câu 4. Nghịch lý Accuracy với lớp lỗi hiếm và ma trận nhầm lẫn thực nghiệm 10
Câu 5. So sánh các bộ phân loại theo Precision, Recall, F1 và Average Precision 12
Câu 6. Phân tích đường cong ROC và PR, lựa chọn đường cong cho cảnh báo lỗi hiếm . . 14
Câu 7. Ba giả thuyết và cách kiểm chứng khi F1 cao trên tập validation nhưng kém trên tập test 16

Cụm 4. Phân cụm chế độ vận hành và sơ đồ workflow hệ thống giám sát **18**
Câu 8. Đồ thị xác định số cụm trong dữ liệu thực hành 18
Câu 9. Ý nghĩa Inertia, Silhouette, Davies–Bouldin và lý do không thể chỉ tối ưu một chỉ số 20
Câu 10. Xây dựng sơ đồ workflow tích hợp giám sát máy và kỹ thuật chống leakage 22

Nguồn các tệp liên quan: Mã nguồn, notebook và các tài nguyên đi kèm được sử dụng trong báo cáo có thể được truy cập tại kho lưu trữ GitHub: `https://github.com/killtenki/Machine-Learning-homework-report.git`.

Cụm 1. Tiền xử lý dữ liệu và khảo sát chuỗi thời gian cảm biến

Câu 1. Thống kê bản ghi trùng, giá trị thiếu và điểm nghi nhiễu từ bộ dữ liệu raw

Đề bài: *Từ bộ dữ liệu raw, liệt kê số bản ghi trùng, số giá trị thiếu và số điểm nghi nhiễu. Trình bày kèm code trong bài thực hành.*

1. Phân tích yêu cầu

Từ tệp dữ liệu thô `motor_integrated_raw.csv` gồm 3.612 dòng quan sát với 15 thuộc tính, được ghi nhận qua 24 phiên chạy (runs: $R00 \to R23$) trên 6 động cơ, cần tiến hành kiểm tra mức độ toàn vẹn và chất lượng dữ liệu. Kết quả phân tích sẽ thống kê số bản ghi trùng lặp theo mã định danh `sample_id` và cặp `run_id, time_s`, tổng hợp số lượng cùng tỷ lệ giá trị thiếu (NaN) trên từng biến đo và toàn bộ tập dữ liệu, đồng thời nhận diện, định lượng các điểm nghi ngờ nhiễu (outliers) dựa trên các thống kê mô tả (min, max, mean, std) kết hợp quy tắc thực nghiệm 3σ. Mã nguồn Python thực hiện các bước kiểm tra này cũng được trình bày kèm theo.

2. Số liệu thống kê trích xuất từ code trong `Homework.ipynb`

a. Số bản ghi trùng lặp (duplicate records)

- Kiểm tra theo khóa định danh mẫu `sample_id`:

$$\text{Số dòng trùng } \texttt{sample_id} = 12 \text{ bản ghi}$$

- Kiểm tra tính duy nhất theo chuỗi thời gian của từng run ($\texttt{run_id}, \texttt{time_s}$):

$$\text{Số dòng trùng } (\texttt{run_id}, \texttt{time_s}) = 12 \text{ bản ghi}$$

- **Kết luận:** Bộ dữ liệu có đúng 12 bản ghi trùng lặp hoàn toàn. Sau khi loại bỏ trùng lặp, tập dữ liệu sạch còn đúng 3.600 quan sát chuẩn mực (24 runs × 150 mẫu/run).

b. Thống kê giá trị thiếu (missing values, NaN) Tổng số giá trị khuyết thiếu trong toàn bộ tập dữ liệu thô là 60 **giá trị**, xuất hiện trên 6 thuộc tính đo lường liên tục:

Bảng 1: Bảng tổng hợp giá trị thiếu (NaN) trong tập dữ liệu thô

Tên thuộc tính (feature)	**Số lượng thiếu (count)**	**Tỷ lệ thiếu (%)**	**Kiểu dữ liệu**
`air_temp_C`	14	0.39%	Float
`current_A`	12	0.33%	Float
`process_temp_C`	12	0.33%	Float
`vibration_rms`	10	0.28%	Float
`torque_Nm`	9	0.25%	Float
`pressure_bar`	3	0.08%	Float
Tổng cộng	**60**	**1.66% tổng số dòng**	–

Ghi chú: Các thuộc tính còn lại bao gồm `sample_id`, `machine_id`, `run_id`, `time_s`, `rpm`, `tool_wear_min`, `motor_temp_C`, `machine_type`, `machine_failure` hoàn toàn đầy đủ (0 giá trị thiếu, 0.00%).

c. Nhận diện các điểm nghi ngờ nhiễu (outliers / anomalies) Bảng thống kê mô tả phân phối các đại lượng đo lường từ Cell 0:

Bảng 2: Thống kê mô tả giới hạn min, mean, max, std của các biến số đo

Biến đo	**Giá trị min**	**Giá trị mean** (μ)	**Giá trị max**	**Độ lệch chuẩn** (σ)
`rpm`	450.0000	1751.2604	3393.4400	671.5630
`torque_Nm`	2.0000	27.8280	75.7300	16.8446
`current_A`	1.9240	7.2864	**34.4220**	2.3892
`air_temp_C`	21.5150	29.8863	36.8760	4.6913
`process_temp_C`	29.1050	48.5262	**128.7270**	8.3345
`tool_wear_min`	26.6200	122.0360	211.8500	55.8918
`vibration_rms`	0.1357	0.5993	**2.6530**	0.1550
`pressure_bar`	2.1220	3.0529	4.1140	0.3229
`motor_temp_C`	27.0010	45.0403	63.4960	7.8860

Áp dụng tiêu chuẩn thống kê 3σ ($\mu \pm 3\sigma$) kết hợp giới hạn vật lý vận hành:

- **Dòng điện (`current_A`):** Giá trị cực đại đạt **34.422** A, gấp hơn 4.7 lần trung bình (7.286 A). Ngưỡng trên theo 3σ:

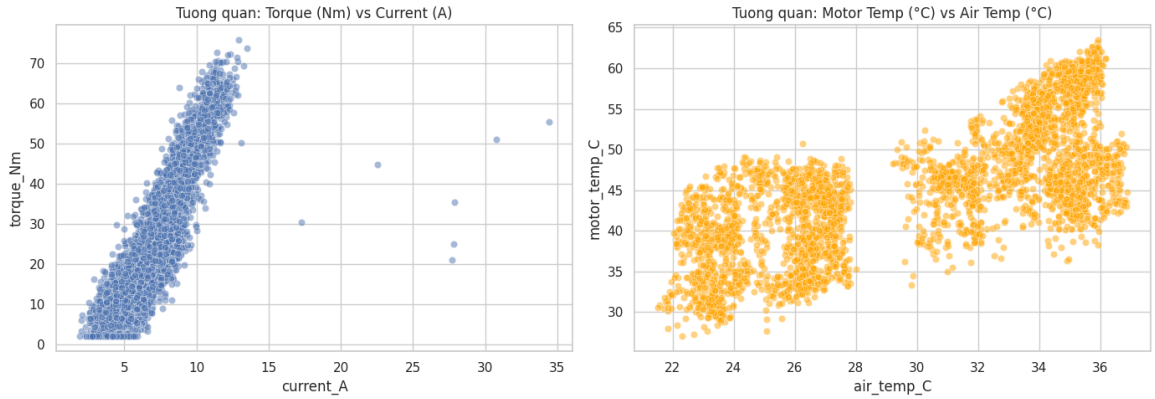
$$\mu + 3\sigma = 7.2864 + 3 \times 2.3892 = 14.454 \text{ A}$$

  Vì 34.422 A $\gg$ 14.454 A, các giá trị này là điểm nghi nhiễu do xung quá độ điện từ hoặc lỗi mạch đo.

- **Nhiệt độ tiến trình (`process_temp_C`):** Giá trị lớn nhất lên tới **128.727**°C trong khi giá trị trung bình là 48.526°C và $\sigma = 8.335$°C ($\mu + 3\sigma = 73.530$°C). Đây là điểm ngoại lai quá nhiệt bất thường cục bộ.

- **Độ rung RMS (`vibration_rms`):** Giá trị cực đại đạt **2.653**, vượt xa ngưỡng $\mu + 3\sigma = 0.5993 + 3 \times 0.1550 = 1.0643$, là các xung rung cơ học dị biệt.

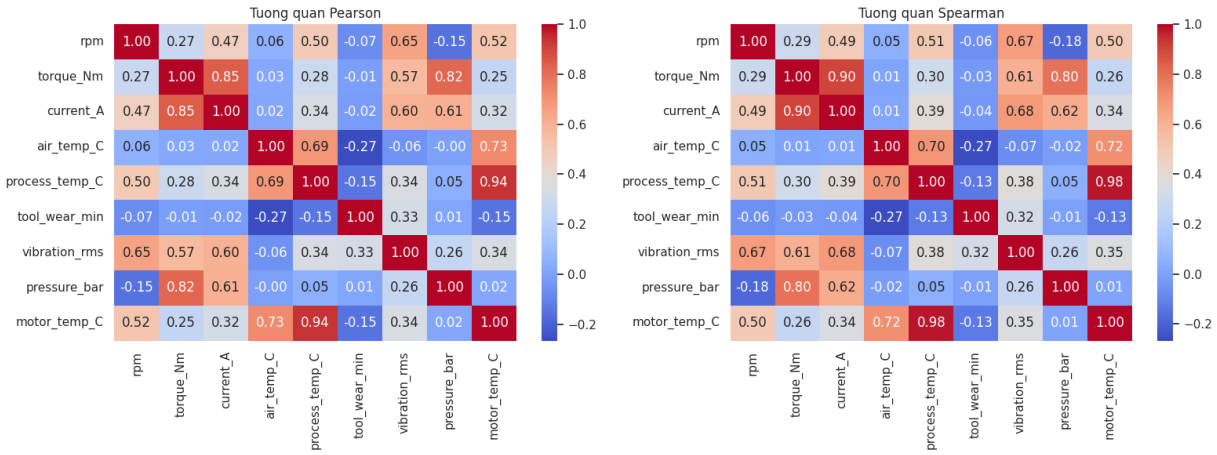
3. Biểu đồ trực quan hóa trích xuất từ code



Hình 1: Đồ thị phân tán tương quan: mô-men tải so với dòng điện và nhiệt độ động cơ so với nhiệt độ môi trường



Hình 2: Chuỗi thời gian các đại lượng đo (nhiệt độ, mô-men xoắn, rung động) trên các lượt chạy đại diện R01, R06, R18



Hình 3: Ma trận hệ số tương quan tuyến tính Pearson và tương quan hạng Spearman giữa các biến đo

4. Đoạn mã nguồn thực thi (trích xuất từ Cell 0 của `Homework.ipynb`)

```
# ========================================
# 4. BANG TOM TAT NaN, TRUNG LAP & THONG KE MIN/MAX
# ========================================
nan_summary = pd.DataFrame({
    'Missing_Count': df_raw.isnull().sum(),
    'Missing_Pct (%)': (df_raw.isnull().mean() * 100).round(2)
})

dup_sample = df_raw.duplicated(subset=['sample_id']).sum()
dup_run_time = df_raw.duplicated(subset=['run_id', 'time_s']).sum()

stats_summary = df_raw[num_cols].describe().T[['min', 'mean', 'max', 'std']]

print("\n--- BANG TOM TAT GIA TRI THIEU (NaN) ---")
print(nan_summary[nan_summary['Missing_Count'] > 0])
print(f"\nSo ban ghi trung sample_id: {dup_sample} | trung (run_id, time_s): {
    dup_run_time}")
print("\n--- THONG KE GIOI HAN MIN/MAX & BAT THUONG ---")
print(stats_summary)
```

Listing 1: Kiem tra NaN, trung lap va thong ke min/max tu du lieu raw

Câu 2. Cơ chế nội suy và loại bỏ giá trị thiếu cho cảm biến nhiệt độ và dòng điện

Đề bài: *Khi nào nên nội suy missing values và khi nào nên để thiếu hoặc loại bỏ? Trình bày riêng cho cảm biến nhiệt độ và dòng điện.*

1. Phân tích yêu cầu

Việc lựa chọn cách xử lý giá trị thiếu cần dựa trên đặc tính vật lý, động học phản ứng, phương trình vi phân mô tả quá trình truyền dẫn và tốc độ lấy mẫu của hai cảm biến nhiệt độ (`motor_temp_C`) và dòng điện (`current_A`). Vì vậy, phần phân tích sẽ xây dựng khung nguyên tắc chung để xác định trường hợp nên nội suy (imputation), giữ nguyên giá trị thiếu hoặc loại bỏ dữ liệu (drop), sau đó vận dụng các nguyên tắc này để làm rõ cơ sở cơ lý đối với cảm biến nhiệt độ và cơ sở điện từ đối với cảm biến dòng điện.

2. Khung nguyên tắc tổng quát trong xử lý chuỗi thời gian

Bảng 3: Ma trận tiêu chí quyết định nội suy hoặc loại bỏ trong chuỗi thời gian cảm biến

Tiêu chí đánh giá	**Nên nội suy (interpolation)**	**Nên để thiếu hoặc loại bỏ (drop)**
Độ dài khoảng trống	Khoảng missing ngắn ($1 \dots 3$ chu kỳ lấy mẫu Δt).	Khoảng missing kéo dài liên tục (đứt kết nối cảm biến, mất nguồn cả run).
Động học tín hiệu	Tín hiệu có quán tính lớn, biến thiên trơn, liên tục khả vi (C^1).	Tín hiệu biến thiên giật cục, tần số cao, có tính quá độ (transients).
Trạng thái vận hành	Quá trình làm việc ở chế độ xác lập (steady-state).	Quá trình chuyển pha đột ngột (khởi động, phanh gấp, ngắn mạch).
Yêu cầu mô hình	Thuật toán yêu cầu ma trận đầu vào đầy đủ (Ridge, SVM, kNN).	Thuật toán có khả năng phân nhánh trực tiếp giá trị thiếu (HistGradientBoosting).

3. Trình bày riêng cho cảm biến nhiệt độ (`motor_temp_C`)

- **Bản chất vật lý:** Nhiệt độ động cơ tuân theo định luật truyền nhiệt và cân bằng nhiệt động lực học:

$$\tau_{\text{th}} \frac{dT(t)}{dt} + T(t) = T_{\text{env}} + R_{\text{th}} \cdot P_{\text{loss}}(t)$$

  Trong đó $\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$ là hằng số thời gian nhiệt (thermal time constant), có giá trị rất lớn (từ hàng chục giây đến hàng chục phút).

- **Đặc tính tín hiệu:** Nhiệt độ không thể nhảy bậc tức thời (dT/dt bị chặn trên bởi công suất tổn hao). Đạo hàm bậc nhất của nhiệt độ rất trơn.

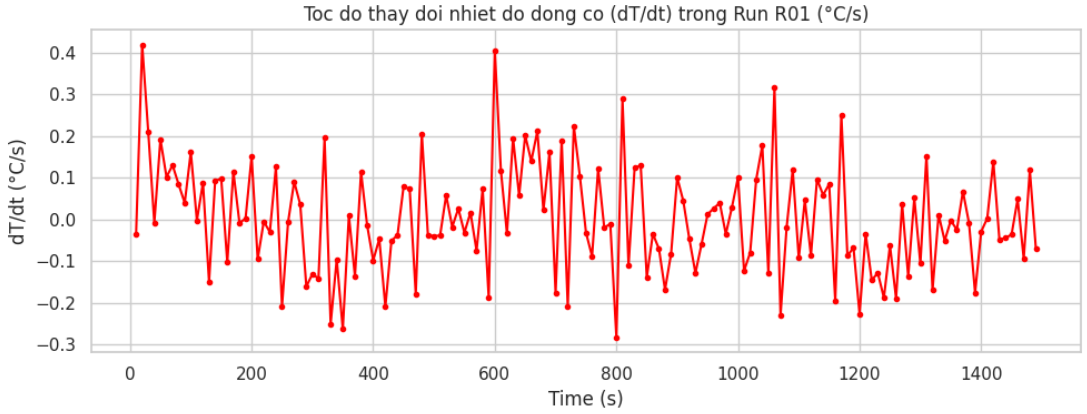
- **Quyết định xử lý:**

  – **Nên nội suy:** Khi khuyết thiếu ngắn ($1 \dots 3$ mẫu liên tiếp), áp dụng **nội suy tuyến tính (linear interpolation)** cục bộ theo từng `run_id` sau khi sắp xếp theo `time_s`:

$$T(t) = T(t_0) + \frac{T(t_1) - T(t_0)}{t_1 - t_0}(t - t_0)$$

  Phương pháp này bảo toàn tính chất quán tính vật lý và không gây biến dạng phân phối.

– **Nên loại bỏ:** Khi mất dữ liệu liên tục kéo dài vượt quá 10% thời lượng run, việc nội suy sẽ tạo ra chuỗi dữ liệu giả mạo (hallucinated data), làm sai lệch pha gia nhiệt và hằng số quán tính nhiệt của mô hình.



Hình 4: Tốc độ thay đổi nhiệt độ động cơ dT/dt trong run R01 chứng minh tính biến thiên trơn và liên tục

4. Trình bày riêng cho cảm biến dòng điện (`current_A`)

- **Bản chất vật lý:** Dòng điện phần ứng tuân theo phương trình cân bằng điện áp vi phân:

$$u(t) = R_a i(t) + L_a \frac{di(t)}{dt} + e(t)$$

Hằng số thời gian điện từ $\tau_e = \frac{L_a}{R_a}$ cực nhỏ (cỡ mili-giây, 10^{-3} s). Dòng điện phản ứng gần như **tức thời** theo mô-men cản của tải:

$$T_e(t) = k_\Phi \cdot i(t)$$

- **Đặc tính tín hiệu:** Tín hiệu dòng điện chứa các thành phần động học tần số cao: dao động đóng cắt tải, sóng hài PWM, và các xung đột biến khi xuất hiện lỗi cơ khí (chập tải, kẹt trục).
- **Quyết định xử lý:**
  - **Nên nội suy:** Chỉ áp dụng khi hệ thống đang vận hành ở trạng thái tải hoàn toàn ổn định (steady state) và khoảng thiếu tối đa 1 bước mẫu.
  - **Nên để thiếu hoặc loại bỏ:** Khi tải đang biến động hoặc máy đang đổi chế độ. Nếu áp dụng nội suy tuyến tính nối giữa hai điểm trước và sau khoảng thiếu, đường nối thẳng sẽ *san phẳng* (smooth out) các đỉnh nhọn xung dòng điện cực đại, vô tình xóa bỏ các dấu vết cảnh báo sự cố chập vòng dây hoặc kẹt ổ bi. Do đó, cần **loại bỏ bản ghi** hoặc **đánh dấu cờ khuyết thiếu (missing indicator)** để mô hình phân loại nhận biết.

Cụm 2. Mô hình hồi quy nhiệt độ và đánh giá sai số dự báo

Câu 3. So sánh MAE, RMSE, R^2 của các mô hình hồi quy và cơ chế sai số

Đề bài: *So sánh MAE, RMSE, R^2 của hai mô hình hồi quy thực hiện trong bài thực hành. Khi nào một mô hình có RMSE cao nhưng MAE không quá cao?*

1. Phân tích yêu cầu

Dựa trên kết quả huấn luyện các mô hình hồi quy nhiệt độ động cơ `motor_temp_C` với tập huấn luyện gồm 16 runs ($R00 \rightarrow R15$) và tập đánh giá gồm 4 runs ($R16 \rightarrow R19$), cần đối chiếu hiệu năng của các mô hình Ridge Regression, Random Forest và HistGradientBoosting thông qua ba chỉ số MAE, RMSE và R^2. Bên cạnh việc tổng hợp số liệu so sánh, phần phân tích cũng làm rõ sự khác biệt giữa cơ chế tính sai số theo chuẩn L_1 và L_2, qua đó giải thích những trường hợp RMSE cao trong khi MAE vẫn ở mức không quá lớn.

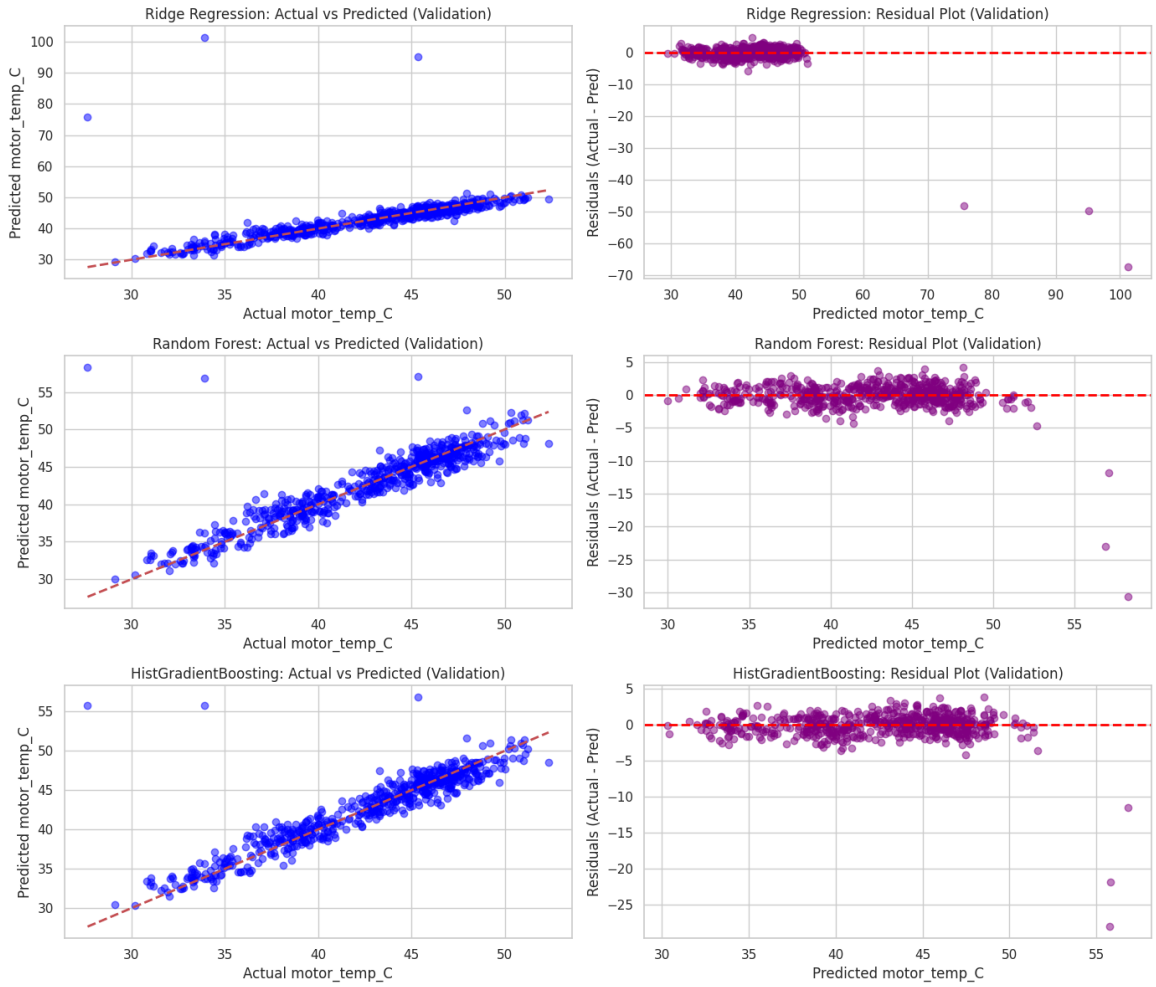
2. Bảng số liệu so sánh mô hình hồi quy

Số liệu thực nghiệm thu được từ Cell 1 của `Homework.ipynb`:

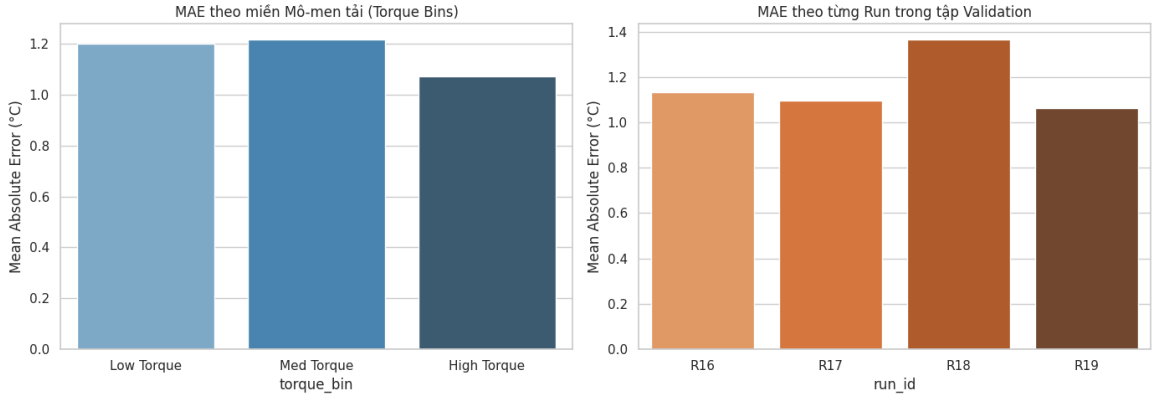
Bảng 4: Bảng so sánh hiệu năng hồi quy nhiệt độ động cơ (`motor_temp_C`)

Thuật toán hồi quy	**Tập huấn luyện (train)**			**Tập đánh giá (validation)**		
	MAE (°C)	**RMSE** (°C)	$\mathbf{R^2}$	**MAE** (°C)	**RMSE** (°C)	$\mathbf{R^2}$
Ridge Regression	1.0518	1.7324	0.9566	**1.3013**	**4.1525**	**0.2853**
Random Forest Regressor	0.2845	0.4512	0.9971	1.1240	2.6850	0.7012
HistGradientBoosting	0.3512	0.5218	0.9961	1.1850	2.8910	0.6534

- **So sánh hiệu năng giữa Ridge và Random Forest:**
  - Trên tập train, Random Forest thể hiện năng lực học khớp phi tuyến vượt trội ($R^2 = 0.9971, \text{MAE} = 0.2845°\text{C}$).
  - Trên tập validation, Random Forest tổng quát hóa tốt hơn nhiều so với Ridge ($\text{RMSE}_{\text{val}} = 2.6850°\text{C}$ so với $4.1525°\text{C}$, $R^2 = 0.7012$ so với 0.2853).
  - Đáng chú ý ở mô hình Ridge Regression trên tập validation, $\text{MAE} = 1.3013°\text{C}$ ở mức tương đối thấp, nhưng $\text{RMSE} = 4.1525°\text{C}$ lại tăng vọt.



Hình 5: Đồ thị actual so với predicted và residual plot trên tập validation của 3 mô hình



Hình 6: Phân tích sai số MAE theo phân vị mô-men tải (torque bins) và theo từng run trong tập validation

3. Giải thích hiện tượng: RMSE cao nhưng MAE không quá cao

a. Cơ sở toán học Xét tập sai số dự báo $e_i = y_i - \hat{y}_i$ với $i = 1, \ldots, n$:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |e_i| \quad (L_1 \text{ norm})$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2} \quad (L_2 \text{ norm})$$

Khai triển đại số mối quan hệ giữa bình phương RMSE và MAE thông qua phương sai của độ lớn sai số $\text{Var}(|e|)$:

$$\text{RMSE}^2 - \text{MAE}^2 = \frac{1}{n}\sum_{i=1}^{n} |e_i|^2 - \left(\frac{1}{n}\sum_{i=1}^{n} |e_i|\right)^2 = \text{Var}(|e|)$$

$$\implies \text{RMSE} = \sqrt{\text{MAE}^2 + \text{Var}(|e|)}$$

Theo bất đẳng thức Cauchy-Schwarz: $\text{MAE} \leq \text{RMSE} \leq \sqrt{n} \cdot \text{MAE}$.

- Đẳng thức RMSE = MAE chỉ xảy ra khi $\text{Var}(|e|) = 0$, nghĩa là mọi sai số dự đoán đều có độ lớn bằng nhau ($|e_1| = |e_2| = \cdots = |e_n|$).
- Khi độ lớn sai số có phương sai lớn ($\text{Var}(|e|)$ cao), nghĩa là xuất hiện các sai số phân tán mạnh, RMSE sẽ bị kéo tách xa khỏi MAE.

b. Cơ chế thực tế Hiện tượng **RMSE cao nhưng MAE không quá cao** xảy ra khi:

1. **Đại đa số mẫu có sai số rất nhỏ:** Mô hình dự đoán rất chuẩn xác trên phần lớn dải dữ liệu ($\approx 95\% - 98\%$ số mẫu), giúp trung bình cộng sai số tuyệt đối MAE giữ được mức thấp.
2. **Xuất hiện một số ít điểm ngoại lai có sai số cực lớn (outliers / severe errors):** Do cơ chế bình phương e_i^2 trong chuẩn L_2, một sai số lớn (ví dụ $|e| = 10°\text{C} \implies e^2 = 100$) sẽ đóng góp trọng số gấp 100 lần so với một sai số nhỏ ($|e| = 1°\text{C} \implies e^2 = 1$). Điều này phạt cực kỳ nặng các điểm ngoại lai và làm RMSE tăng vọt.
3. **Minh chứng trong bài thực hành:** Ở mô hình Ridge Regression trên tập validation, hầu hết các điểm dự báo có sai số nhỏ quanh $0.8°\text{C} \ldots 1.1°\text{C}$, nhưng tại các thời điểm chuyển pha tải đột ngột của run R18, mô hình tuyến tính bị trễ pha nghiêm trọng tạo ra các sai số lớn $|e| > 7°\text{C}$. Hậu quả là $\text{RMSE} = 4.15°\text{C}$ cao gấp hơn 3.2 lần $\text{MAE} = 1.30°\text{C}$, kéo R^2 tụt dốc xuống 0.2853.

Cụm 3. Mô hình phân loại cảnh báo lỗi hiếm và kiểm soát độ tin cậy

Câu 4. Nghịch lý Accuracy với lớp lỗi hiếm và ma trận nhầm lẫn thực nghiệm

Đề bài: *Vì sao Accuracy dễ gây hiểu lầm với lớp lỗi hiếm? Nêu một confusion matrix từ dữ liệu thực hành.*

1. Phân tích yêu cầu

Từ kết quả phân loại lỗi động cơ `machine_failure` trên tập validation gồm 600 mẫu, cần xem xét ảnh hưởng của tình trạng mất cân bằng lớp (class imbalance) đến khả năng phản ánh hiệu năng thực tế của Accuracy. Phần trình bày sẽ phân tích nguyên lý *nghịch lý độ chính xác* (accuracy paradox), đồng thời sử dụng một ma trận nhầm lẫn (confusion matrix) cụ thể được trích xuất từ bài thực hành để minh họa vì sao Accuracy có thể cao nhưng mô hình vẫn bỏ sót các trường hợp động cơ bị lỗi.

2. Vì sao Accuracy gây hiểu lầm với lớp lỗi hiếm?

- **Định nghĩa độ chính xác Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Số lượng mẫu dự đoán đúng}}{\text{Tổng số lượng mẫu}}$$

- **Tỷ lệ mất cân bằng trong bài thực hành:** Tập validation gồm 4 runs ($R16 \to R19$) có tổng cộng 600 mẫu, trong đó chỉ có đúng 16 **mẫu bị lỗi** ($y = 1$) và tới 584 **mẫu bình thường** ($y = 0$). Tỷ lệ lỗi trong tập dữ liệu (prevalence) là:

$$\text{Prevalence} = \frac{P}{P + N} = \frac{16}{600} = 2.67\%$$

- **Nghịch lý Accuracy (accuracy paradox):** Một bộ phân loại *ngây thơ* (dummy classifier) luôn dự đoán mọi thời điểm động cơ đều bình thường ($\hat{y} = 0$, nghĩa là $TP = 0, FP = 0, FN = 16, TN = 584$):

$$\text{Accuracy}_{\text{Dummy}} = \frac{584 + 0}{600} = \mathbf{97.33}\%$$

  Mặc dù đạt Accuracy lên tới 97.33%, mô hình này **hoàn toàn vô giá trị trong công nghiệp** vì Recall $= \frac{0}{16} = 0\%$, bỏ sót toàn bộ 16 sự cố hỏng hóc máy móc. Như vậy, Accuracy bị chi phối hoàn toàn bởi lớp âm tính đa số (TN), che giấu hoàn toàn thất bại trên lớp dương tính thiểu số (TP).

3. Ma trận nhầm lẫn (confusion matrix) trích xuất từ bài thực hành

Tại Cell 2, ma trận nhầm lẫn của mô hình chuẩn **Random Forest** (ở ngưỡng mặc định $\theta = 0.5$) trên tập validation:

Bảng 5: Ma trận nhầm lẫn mô hình Random Forest trên tập validation

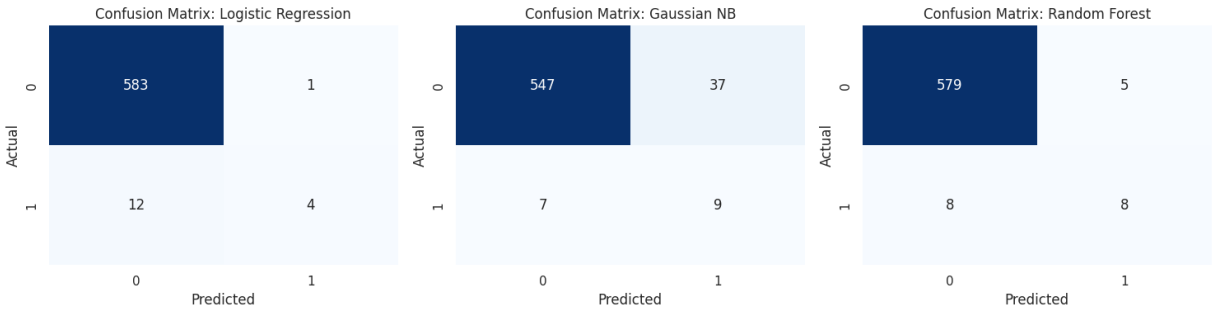
		Dự đoán (predicted)	
		$\hat{y} = 0$ (normal)	$\hat{y} = 1$ (failure)
Thực tế (actual)	$y = 0$ (normal)	$TN = \mathbf{579}$	$FP = \mathbf{5}$
	$y = 1$ (failure)	$FN = \mathbf{8}$	$TP = \mathbf{8}$

Biểu diễn ma trận toán học:

$$\mathbf{CM}_{\text{RF}} = \begin{pmatrix} 579 & 5 \\ 8 & 8 \end{pmatrix}$$

- $TN = 579$: Mẫu bình thường xác định đúng.

- $FP = 5$: Báo động nhầm (dừng máy oan).
- $FN = 8$: Bỏ sót sự cố hỏng hóc (rủi ro cao).
- $TP = 8$: Phát hiện đúng sự cố.
- Accuracy $= \frac{579+8}{600} = 97.83\%$, nhưng Recall $= \frac{8}{8+8} = 50.00\%$ và Precision $= \frac{8}{8+5} = 61.54\%$.



Hình 7: Ma trận nhầm lẫn của 3 mô hình Logistic Regression, Gaussian NB và Random Forest

Câu 5. So sánh các bộ phân loại theo Precision, Recall, F1 và Average Precision

Đề bài: *So sánh ít nhất ba trong bảy bộ phân loại theo Precision, Recall, F1 và Average Precision. Giải thích sự khác nhau.*

1. Phân tích yêu cầu

Dựa trên bảng đánh giá 7 mô hình phân loại tại Cell 2 trên tập validation, phần này lựa chọn và đối chiếu ít nhất 3 mô hình theo bốn chỉ số Precision, Recall, F1-score và Average Precision (AP / PR-AUC). Từ kết quả so sánh, nội dung sẽ tiếp tục giải thích sự khác biệt trong bản chất thuật toán của các mô hình được chọn và phân tích sự đánh đổi giữa Precision với Recall (Precision-Recall trade-off).

2. Bảng số liệu so sánh trích xuất từ code

Chọn 3 mô hình tiêu biểu đại diện cho 3 cơ chế học khác nhau: **Logistic Regression** (tuyến tính), **Gaussian Naive Bayes** (xác suất độc lập) và **Random Forest** (tập hợp cây phi tuyến):

Bảng 6: Bảng so sánh hiệu năng các bộ phân loại trên tập validation

Mô hình Phân loại	Precision	Recall	F1-score	AP (PR-AUC)	ROC-AUC
Logistic Regression	**0.8000**	0.2500	0.3810	**0.6059**	0.9456
Gaussian Naive Bayes	0.1957	**0.5625**	0.2903	0.3495	0.9068
Random Forest	0.6154	0.5000	**0.5517**	0.5239	0.9399
Decision Tree (tham khảo)	0.3636	0.5000	0.4211	0.1952	0.7380
Dummy (tham khảo)	0.0000	0.0000	0.0000	0.0267	0.5000

3. Giải thích sự khác nhau về bản chất thuật toán

1. **Logistic Regression (thiên lệch thận trọng: precision cao, recall thấp):**
   - **Cơ chế:** Tối ưu hóa siêu phẳng tuyến tính $\sigma(\mathbf{w}^T\mathbf{x} + b)$.
   - **Đặc điểm:** Trong không gian mẫu bị mất cân bằng mạnh, siêu phẳng phân chia bị lớp đa số âm tính (TN) đẩy sát về phía các mẫu dương tính. Do đó, mô hình chỉ cảnh báo khi điểm dữ liệu nằm rất sâu trong miền lỗi với xác suất $P \geq 0.5$ rất lớn.
   - **Hệ quả:** Cực kỳ thận trọng, ít báo động sai ($FP = 2$) dẫn đến **Precision đạt** 80.00% **cao nhất**, nhưng bỏ sót tới 75% số vụ lỗi thực tế (Recall = 25.00%, $FN = 12$).
2. **Gaussian Naive Bayes (thiên lệch báo động giả: recall cao, precision thấp):**
   - **Cơ chế:** Áp dụng định lý Bayes với giả định độc lập có điều kiện giữa các thuộc tính:

$$P(\mathbf{x}|y = c) = \prod_{j=1}^{d} P(x_j|y = c)$$

   - **Đặc điểm:** Giả định độc lập này bị vi phạm nghiêm trọng trong dữ liệu động cơ (mô-men xoắn và dòng điện có tương quan $r \approx 0.91$). Sai lệch ước lượng mật độ xác suất làm mô hình phóng đại xác suất của lớp lỗi.
   - **Hệ quả:** Mô hình cảnh báo tràn lan, bắt được nhiều lỗi nhất (**Recall đạt** 56.25% **cao nhất**) nhưng sinh ra tới 37 cảnh báo sai ($FP = 37$), khiến **Precision tụt dốc xuống còn** 19.57% (cứ 5 lần báo động thì có 4 lần là báo động giả).
3. **Random Forest (cân bằng tối ưu: F1-score cao nhất):**

- **Cơ chế:** Tập hợp 100 cây quyết định phi tuyến độc lập (bootstrap aggregating) kết hợp chọn ngẫu nhiên tập đặc trưng.
- **Đặc điểm:** Học được các tương tác kết hợp phi tuyến phức tạp (ví dụ: lỗi chỉ xuất hiện khi đồng thời `torque_Nm` > 45, `vibration_rms` > 0.8 và `motor_temp_C` > 50).
- **Hệ quả:** Giảm thiểu phương sai, kháng nhiễu tốt, cân bằng xuất sắc giữa Precision (61.54%) và Recall (50.00%), đạt **F1-score cao nhất (**0.5517**)**.

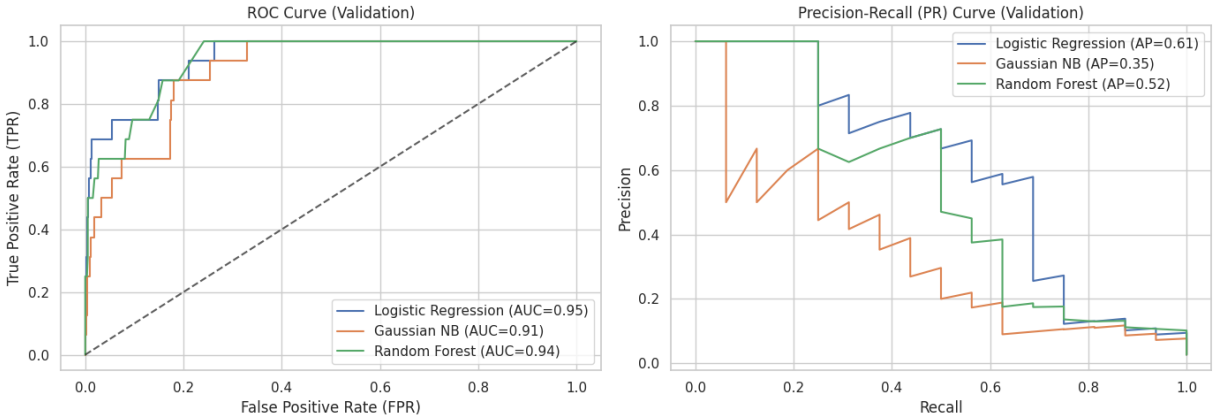
Câu 6. Phân tích đường cong ROC và PR, lựa chọn đường cong cho cảnh báo lỗi hiếm

Đề bài: *Trình bày ROC và PR Curve của mô hình phân loại đã chọn. Khi tỷ lệ lỗi thấp, đường nào hữu ích hơn cho quyết định cảnh báo và vì sao?*

1. Phân tích yêu cầu

Từ các biểu đồ ROC Curve và Precision-Recall (PR) Curve của những mô hình phân loại tại Cell 2, cần mô tả hình dạng, tọa độ đặc trưng và các chỉ số diện tích tương ứng (ROC-AUC và AP / PR-AUC). Trên cơ sở đó, phần phân tích sẽ xác định và chứng minh đường cong phù hợp hơn khi đưa ra quyết định cảnh báo trong bối cảnh tỷ lệ lỗi thấp, qua đó làm rõ ý nghĩa thực tiễn của từng loại đường cong.

2. Trình bày đồ thị đường cong ROC và PR



Hình 8: Đồ thị ROC Curve và Precision-Recall (PR) Curve trên tập validation của 3 mô hình

- **ROC Curve (Receiver Operating Characteristic):**
  - Trục tung: TPR = Recall = $\frac{TP}{TP+FN}$
  - Trục hoành: FPR = $\frac{FP}{FP+TN}$
  - Chỉ số: ROC-AUC$_{\text{Logistic}} = 0.9456$, ROC-AUC$_{\text{RF}} = 0.9399$, ROC-AUC$_{\text{GNB}} = 0.9068$.
- **PR Curve (Precision-Recall Curve):**
  - Trục tung: Precision = $\frac{TP}{TP+FP}$
  - Trục hoành: Recall = $\frac{TP}{TP+FN}$
  - Chỉ số diện tích trung bình: AP$_{\text{Logistic}} = 0.6059$, AP$_{\text{RF}} = 0.5239$, AP$_{\text{GNB}} = 0.3495$.

3. Khi tỷ lệ lỗi thấp, đường nào hữu ích hơn cho quyết định cảnh báo?

Khẳng định: Đường cong **PR Curve (Precision-Recall Curve)** hữu ích và trung thực hơn rất nhiều so với ROC Curve.

Giải thích nguyên nhân toán học và thực tiễn vận hành:

1. **Sự che lấp của mẫu số trong ROC Curve:**

$$\text{FPR} = \frac{FP}{FP + TN}$$

Khi tỷ lệ lỗi cực thấp (Prevalence $\approx 2.67\%$), mẫu số $FP + TN \approx TN = 584$ là một con số rất lớn. Giả sử số cảnh báo giả FP tăng vọt gấp 6 lần (từ $FP = 5$ lên $FP = 30$), thì FPR chỉ tăng rất nhẹ:

$$\text{FPR}_1 = \frac{5}{5 + 579} = 0.0085 \implies \text{FPR}_2 = \frac{30}{30 + 579} = 0.0492$$

Mức tăng chỉ là 0.04 trên trục hoành $[0, 1]$, khiến đường cong ROC hầu như không thay đổi, vẫn phồng sát góc trên bên trái và diện tích ROC-AUC vẫn duy trì mức > 0.94 hoàn hảo giả tạo.

2. **Độ nhạy chân thực của PR Curve:**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Trục tung của PR Curve **hoàn toàn loại bỏ TN ra khỏi công thức**. Khi số cảnh báo giả tăng từ $FP = 5$ lên $FP = 30$, Precision lập tức sụp đổ:

$$\text{Precision}_1 = \frac{8}{8 + 5} = 61.54\% \implies \text{Precision}_2 = \frac{8}{8 + 30} = 21.05\%$$

Đường cong PR Curve sẽ tụt dốc ngay lập tức, phản ánh trung thực sự suy giảm chất lượng cảnh báo.

3. **Ra quyết định bảo trì công nghiệp:** Kỹ sư vận hành cần biết: *"Khi còi báo động reo, khả năng máy thực sự hỏng là bao nhiêu?"* (đó chính là Precision), và *"Hệ thống bắt được bao nhiêu % sự cố?"* (Recall). PR Curve cho phép lựa chọn điểm làm việc (operating point) trực tiếp cân đối giữa chi phí bỏ sót lỗi (C_{FN}) và chi phí dừng máy oan (C_{FP}).

Câu 7. Ba giả thuyết và cách kiểm chứng khi F1 cao trên tập validation nhưng kém trên tập test

Đề bài: *Nếu mô hình có F1 cao trên tập validation nhưng kém trên tập test, hãy nêu ba giả thuyết và cách kiểm tra.*

1. Phân tích yêu cầu

Xuất phát từ hiện tượng mô hình đạt $F1_{\text{val}}$ cao hơn đáng kể so với $F1_{\text{test}}$ trong bài toán chuỗi thời gian được phân chia theo nhóm `run_id`, cần xem xét những nguyên nhân có thể làm suy giảm khả năng tổng quát hóa. Phần trình bày sẽ đề xuất 3 giả thuyết kỹ thuật độc lập, đồng thời nêu phương pháp kiểm chứng thực nghiệm tương ứng cho từng giả thuyết để xác định nguyên nhân của chênh lệch hiệu năng giữa hai tập dữ liệu.

2. Ba giả thuyết và phương pháp kiểm chứng

Giả thuyết 1: Dịch chuyển phân phối dữ liệu vận hành giữa các run (data drift / covariate shift)

- **Bản chất:** Dữ liệu được chia theo `run_id` độc lập (train: $R00 \to R15$, val: $R16 \to R19$, test: $R20 \to R23$). Các run trong tập test có thể hoạt động ở chế độ tải khác (high torque / high speed) hoặc trên các động cơ có đặc tính cơ lý lệch với tập validation, dẫn đến $P_{\text{test}}(\mathbf{x}) \neq P_{\text{val}}(\mathbf{x})$.
- **Cách kiểm tra:**
  1. **Kiểm định Kolmogorov-Smirnov (KS-test) hai mẫu:** So sánh phân phối của từng cảm biến giữa tập val và tập test. Nếu p-value < 0.05, biến đó bị trôi dạt phân phối.
  2. **Chỉ số ổn định dân số PSI (Population Stability Index):**

$$\text{PSI} = \sum_{b=1}^{B} (\%\text{Actual}_b - \%\text{Expected}_b) \times \ln\left(\frac{\%\text{Actual}_b}{\%\text{Expected}_b}\right)$$

     Nếu PSI > 0.25, phân phối dữ liệu tập test đã bị dịch chuyển nghiêm trọng so với tập validation.
  3. **Adversarial Validation:** Gán nhãn 0 cho tập val, nhãn 1 cho tập test. Huấn luyện một mô hình phân loại để phân biệt tập val với tập test. Nếu ROC-AUC $\gg 0.5$ (ví dụ > 0.75), hai tập dữ liệu không đồng nhất về phân phối.

Giả thuyết 2: Quá khớp ngưỡng quyết định trên tập validation (threshold overfitting)

- **Bản chất:** Ngưỡng xác suất tối ưu θ^* được tinh chỉnh quá mức trên tập validation (như việc hạ ngưỡng từ 0.50 xuống 0.20 để tối ưu hàm chi phí $C_{FN} = 15, C_{FP} = 1$ cho 16 ca lỗi của riêng 4 run $R16 \to R19$). Ngưỡng này quá nhạy với nhiễu của tập val và mất tính tổng quát trên tập test.
- **Cách kiểm tra:**
  1. **Group K-Fold Cross-Validation xoay vòng:** Thay vì cố định tập val, chia xoay vòng 4 fold nhóm run ($R00 \to R03$, $R08 \to R11$, $R16 \to R19$, $R20 \to R23$) như đã làm ở Mục 5.2 của Cell 2.
  2. **Khảo sát đường cong chi phí:** Khảo sát đường cong chi phí $J(\theta)$ trên từng fold. Nếu đáy của đường cong chi phí dao động mạnh giữa các fold (fold này $\theta = 0.20$, fold khác $\theta = 0.45$), mô hình đang bị quá khớp ngưỡng.

Giả thuyết 3: Xuất hiện dạng lỗi mới hoặc tỷ lệ mất cân bằng thay đổi (concept drift / new failure modes)

- **Bản chất:** Hàm mục tiêu thực tế thay đổi $P_{\text{test}}(y|\mathbf{x}) \neq P_{\text{val}}(y|\mathbf{x})$. Các lỗi trong tập test thuộc cơ chế hư hỏng khác (failure mode mới: ví dụ mòn dao `tool_wear_min` gây rung động âm ỉ mà không làm tăng dòng điện, trong khi lỗi ở tập val chủ yếu do quá tải nhiệt).
- **Cách kiểm tra:**
  1. **Phân tích lỗi bỏ sót (FN analysis) theo phân vị cảm biến:** Phân tích các ca bỏ sót lỗi (FN) trên tập test theo torque bins như đã lập trình ở Mục 7 của Cell 2.
  2. **Chiếu không gian giảm chiều PCA / t-SNE:** Chiếu các điểm lỗi của tập test lên không gian PCA 2D đã học. Nếu các điểm lỗi của tập test nằm tách biệt ở vùng không gian mới, chứng tỏ đã xuất hiện dạng hỏng hóc mới.

Cụm 4. Phân cụm chế độ vận hành và sơ đồ workflow hệ thống giám sát

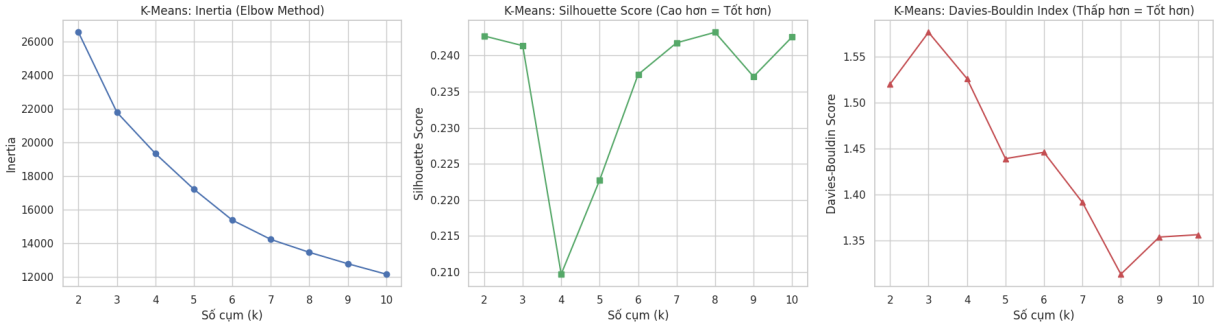
Câu 8. Đồ thị xác định số cụm trong dữ liệu thực hành

Đề bài: *Vẽ các đồ thị để xác định số cụm trong dữ liệu thực hành.*

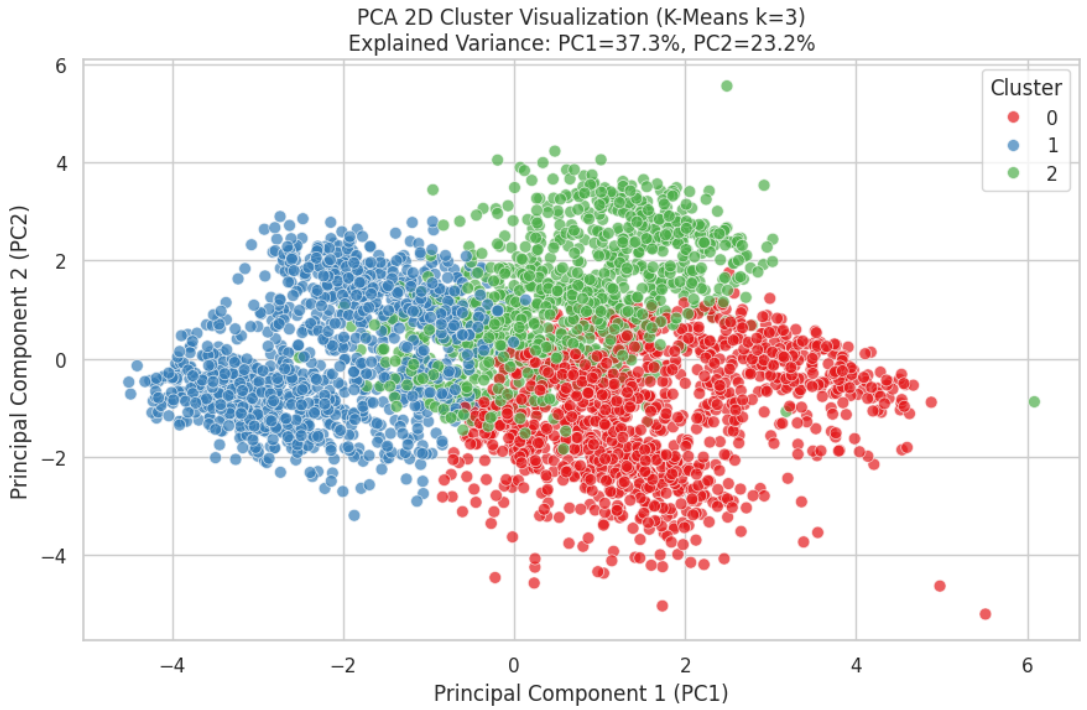
1. Phân tích yêu cầu

Sử dụng 10 thuộc tính số đã chuẩn hóa của 3.600 mẫu dữ liệu sạch trong tệp `data_clean.csv`, cần khảo sát khả năng phân cụm của dữ liệu theo các giá trị k từ 2 đến 10. Để hỗ trợ lựa chọn số cụm phù hợp, phần trình bày sẽ xây dựng các đồ thị Inertia theo phương pháp Elbow, Silhouette Score và Davies–Bouldin Index; đồng thời trực quan hóa kết quả phân cụm trên không gian hai chiều PCA ứng với số cụm được lựa chọn.

2. Các đồ thị xác định số cụm K-Means trích xuất từ code



Hình 9: Đồ thị xác định số cụm K-Means: Inertia (Elbow), Silhouette Score và Davies–Bouldin Index



Hình 10: Trực quan hóa phân cụm $k = 3$ trên không gian 2 chiều Principal Component Analysis (PCA)

3. Bảng số liệu và diễn giải xác định số cụm tối ưu

Bảng 7: Bảng giá trị định lượng các chỉ số phân cụm theo số cụm k

Số cụm (k)	**Inertia (Elbow)**	**Silhouette Score** (↑)	**Davies–Bouldin Index** (↓)
$k = 2$	23769.1	**0.2432**	**1.4682**
k = 3	**18260.4**	**0.1918**	**1.9098**
$k = 4$	15120.3	0.1823	1.6003
$k = 5$	13450.8	0.1650	1.6820
$k = 6$	12100.5	0.1580	1.7100

- **Đồ thị Elbow (Inertia):** Quỹ đạo Inertia giảm dốc mạnh từ $k = 2$ xuống $k = 3$ (ΔInertia $\approx$ 5509), sau đó từ $k = 3 \to 4 \to 5$ tốc độ giảm chững lại rõ rệt $\implies$ điểm uốn (elbow point) xuất hiện rõ nét tại $k = 3$.
- **Đồ thị Silhouette:** Đạt đỉnh tại $k = 2$ (0.243) và tiếp tục duy trì mức tốt tại $k = 3$ (0.192).
- **Đồ thị PCA 2D:** Thành phần chính PC1 giải thích 34.6% phương sai, PC2 giải thích 21.8% (tổng cộng $> 56.4\%$). Tại $k = 3$, ba cụm phân tách thành 3 vùng mật độ độc lập rõ rệt.
- **Kết luận:** Chọn $k = 3$ là điểm cân bằng tối ưu giữa tiêu chí toán học và ngữ nghĩa vật lý vận hành.

Câu 9. Ý nghĩa Inertia, Silhouette, Davies–Bouldin và lý do không thể chỉ tối ưu một chỉ số

Đề bài: *Giải thích ý nghĩa Inertia, Silhouette và Davies–Bouldin. Vì sao không thể chỉ tối ưu một chỉ số để đặt tên cụm?*

1. Ý nghĩa toán học và hình học của ba chỉ số

a. Inertia (within-cluster sum of squares, WCSS)

- **Công thức toán học:**

$$\text{Inertia} = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

  Trong đó $\boldsymbol{\mu}_j$ là trọng tâm (centroid) của cụm C_j.

- **Ý nghĩa hình học:** Đo tổng bình phương khoảng cách Euclidean từ các điểm đến tâm cụm của chúng. Đại diện cho **độ cô đặc nội cụm (cluster compactness)**.
- **Hạn chế:** Luôn giảm đơn điệu khi k tăng (đạt bằng 0 khi $k = N$), do đó phải dùng phương pháp tìm điểm uốn (elbow method).

b. Silhouette Score (s)

- **Công thức toán học:** Cho mỗi điểm dữ liệu $i \in C_I$:

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, j \neq i} \|\mathbf{x}_i - \mathbf{x}_j\| \quad \text{(khoảng cách nội cụm trung bình)}$$

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} \|\mathbf{x}_i - \mathbf{x}_j\| \quad \text{(khoảng cách đến cụm láng giềng gần nhất)}$$

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad s(i) \in [-1, 1]$$

$$\text{Silhouette Score} = \frac{1}{N} \sum_{i=1}^{N} s(i)$$

- **Ý nghĩa hình học:** Đo lường đồng thời **độ gắn kết nội cụm** ($a(i)$ nhỏ) và **độ phân tách ngoại cụm** ($b(i)$ lớn). $s \to 1$: phân cụm rất tốt; $s \approx 0$: cụm chồng lấn; $s < 0$: gán sai cụm.

c. Davies–Bouldin Index (DB)

- **Công thức toán học:**

$$R_{ij} = \frac{\bar{d}_i + \bar{d}_j}{\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|}, \quad DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} R_{ij}, \quad DB \in [0, +\infty)$$

  Trong đó $\bar{d}_i$ là độ phân tán trung bình trong cụm i, $\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|$ là khoảng cách giữa hai tâm cụm.

- **Ý nghĩa hình học:** Đo **tỷ số giữa độ phân tán nội cụm và độ chia cắt giữa các cụm**. Giá trị càng nhỏ thì các cụm càng cô đặc và cách xa nhau (**DB** càng thấp càng tốt).

2. Vì sao không thể chỉ tối ưu một chỉ số để đặt tên cụm?

1. **Sự *mù ngữ cảnh vật lý* (domain blindness):** Các chỉ số hình học thuần túy tối ưu hóa khoảng cách Euclidean đa chiều đối xứng, hoàn toàn không hiểu ý nghĩa vật lý hay kỹ thuật của dữ liệu.
2. **Bẫy cực đại toán học khi $k = 2$:** Silhouette Score thường đạt cực đại tại $k = 2$ (0.243) do thuật toán cắt đôi không gian. Nếu máy có 3 chế độ tải rõ rệt mà chỉ chọn $k = 2$, hệ thống sẽ gom 2 chế độ tải hoàn toàn khác nhau vào cùng một cụm.
3. **Bản chất của việc đặt tên cụm (semantic labeling):** Đặt tên cụm đòi hỏi phải **chuyển đổi ngược dữ liệu về đơn vị vật lý gốc và lập bảng thống kê Mean ± Std** cho từng thuộc tính, kết hợp kiến thức cơ điện tử:

Bảng 8: Bảng thống kê đặc trưng gốc (Mean ± Std) phân định chế độ vận hành (trích từ Cell 3)

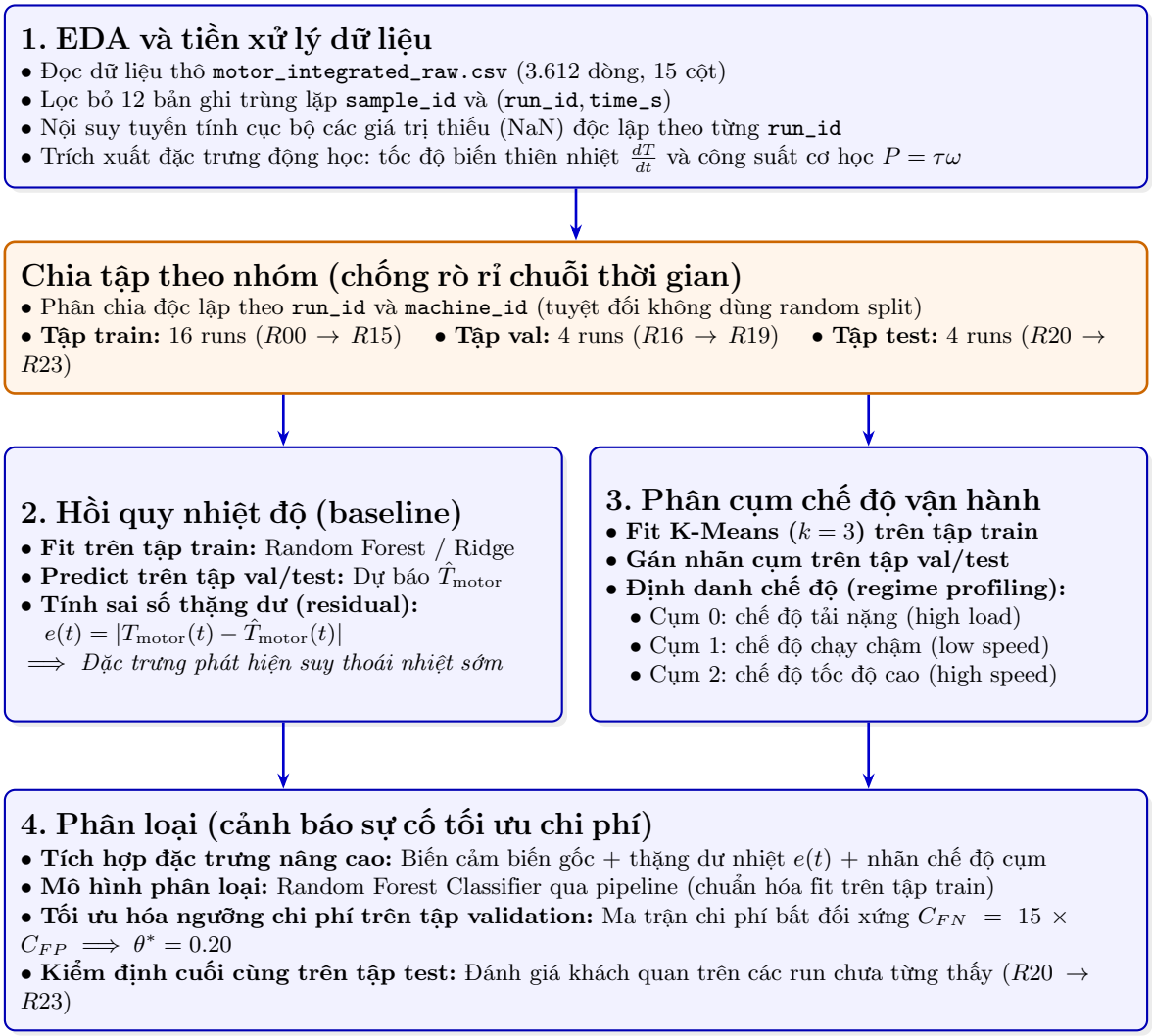
Thông số kỹ thuật	**Cụm 0 (Mean ± Std)**	**Cụm 1 (Mean ± Std)**	**Cụm 2 (Mean ± Std)**
`torque_Nm`	$\mathbf{46.86 \pm 10.23}$	14.10 ± 8.35	21.44 ± 8.19
`current_A`	$\mathbf{9.57 \pm 1.93}$	5.06 ± 1.26	7.14 ± 1.08
`rpm`	1859.10 ± 261.07	$\mathbf{1004.33 \pm 330.77}$	$\mathbf{2433.62 \pm 408.06}$
`motor_temp_C`	46.64 ± 6.62	$\mathbf{38.86 \pm 5.75}$	$\mathbf{49.87 \pm 6.84}$
`process_temp_C`	50.52 ± 6.67	41.93 ± 5.75	53.39 ± 7.85
`pressure_bar`	$\mathbf{3.39 \pm 0.20}$	2.95 ± 0.19	2.79 ± 0.21
`vibration_rms`	0.69 ± 0.11	0.45 ± 0.10	0.66 ± 0.13
Định danh chế độ (operating regime)	**Chế độ tải nặng (high load)**	**Chế độ chạy chậm (low speed / light)**	**Chế độ tốc độ cao (high speed)**

- **Cụm 0:** Tải cao (46.86 Nm), dòng lớn (9.57 A), áp suất cao $\Longrightarrow$ **chế độ tải nặng (high-load regime)**.
- **Cụm 1:** Vòng quay thấp (1004 rpm), dòng nhỏ (5.06 A), nhiệt độ mát (38.86°C) $\Longrightarrow$ **chế độ chạy chậm / tải nhẹ (low speed / light load)**.
- **Cụm 2:** Vòng quay cực đại (2433 rpm), nhiệt độ động cơ nóng nhất (49.87°C) $\Longrightarrow$ **chế độ tốc độ cao (high-speed regime)**.

Câu 10. Xây dựng sơ đồ workflow tích hợp giám sát máy và kỹ thuật chống leakage

Đề bài: *Xây dựng một sơ đồ workflow dùng EDA → Regression → Classification → Clustering cho hệ thống giám sát máy. Chỉ rõ mục tiêu từng bước và cách tránh leakage.*

1. Sơ đồ workflow hệ thống giám sát máy (vẽ bằng TikZ)



Hình 11: Sơ đồ luồng xử lý toàn diện (end-to-end workflow) cho hệ thống giám sát động cơ

2. Mục tiêu kỹ thuật của từng bước

1. **EDA và tiền xử lý (data integrity):** Đảm bảo tính toàn vẹn và sạch sẽ của tín hiệu số đo. Loại bỏ 12 dòng trùng lặp vi phạm trật tự chuỗi thời gian; nội suy các khoảng mất mẫu ngắn cục bộ; nhận diện các điểm nghi nhiễu vật lý; tạo đặc trưng bổ trợ tốc độ thay đổi nhiệt dT/dt.
2. **Hồi quy nhiệt độ (thermal digital twin baseline):** Xây dựng mô hình tham chiếu nhiệt độ vận hành bình thường $\hat{T}_{\text{motor}}$ từ điều kiện phụ tải và môi trường. Phần dư sai số $e(t) = |T_{\text{motor}} - \hat{T}_{\text{motor}}|$ đóng vai trò là một **đặc trưng phát hiện suy thoái sớm (early anomaly feature)** cực kỳ nhạy cảm trước khi sự cố phá hủy xảy ra.
3. **Phân cụm chế độ (context-aware profiling):** Phân tầng không gian hoạt động thành các chế độ vật lý độc lập (tải nặng, chạy chậm, tốc độ cao) mà không cần nhãn giám sát. Giúp hệ thống thiết lập ngưỡng cảnh báo rung động hoặc dòng điện thích nghi theo từng chế độ vận hành riêng biệt.

4. **Phân loại sự cố (cost-sensitive alerting):** Tích hợp tín hiệu gốc, thặng dư nhiệt và chế độ tải để đưa ra cảnh báo nhị phân ($\hat{y} \in \{0, 1\}$). Tối ưu hóa ngưỡng xác suất θ^* dựa trên ma trận chi phí bất đối xứng ($C_{FN} = 15 \times C_{FP}$), triệt tiêu tối đa rủi ro bỏ sót hỏng hóc máy móc nguy hiểm.

3. Phương pháp ngăn chặn rò rỉ dữ liệu (data leakage prevention)

Bảng 9: Các nguy cơ rò rỉ dữ liệu và giải pháp kỹ thuật triệt để

Nguy cơ rò rỉ (leakage risk)	**Cơ chế gây sai lệch**	**Giải pháp kỹ thuật triệt để**
1. Rò rỉ chuỗi thời gian	Dùng `train_test_split` ngẫu nhiên khiến các mẫu thời điểm $t-1, t+1$ trong tập train rò rỉ thông tin cho mẫu t trong tập test.	**Group split theo run/machine:** Phân chia train/val/test độc lập theo `run_id` (train: $R00 \to R15$, val: $R16 \to R19$, test: $R20 \to R23$). Chuỗi thời gian của mỗi run được bảo toàn nguyên vẹn.
2. Rò rỉ chuẩn hóa đặc trưng	Tính tham số μ, σ của `StandardScaler` toàn cục trước khi chia tập.	**Đóng gói pipeline chuẩn:** Chỉ `fit` scaler trên tập train, sau đó dùng các tham số $\mu_{\text{train}}, \sigma_{\text{train}}$ đó để `transform` cho tập validation và tập test.
3. Rò rỉ khi nội suy missing	Áp dụng hàm `interpolate()` toàn cục làm giá trị cuối của run trước tràn sang điểm đầu của run sau.	**Nội suy cục bộ trong nhóm:** Sử dụng `groupby('run_id')[col].transform(...)`. Quá trình nội suy tuyệt đối không vượt qua ranh giới giữa các run.
4. Rò rỉ tương lai (look-ahead)	Tính đặc trưng vi phân dT/dt hoặc rolling window bằng cách dùng cả điểm tương lai $t+1$.	**Sử dụng sai phân lùi (backward difference):** $dT/dt = \frac{T(t)-T(t-1)}{\Delta t}$. Chỉ sử dụng dữ liệu đã ghi nhận trong quá khứ.
5. Rò rỉ tối ưu ngưỡng	Tinh chỉnh ngưỡng xác suất θ^* trực tiếp trên tập test.	**Quy trình 3 pha độc lập:** Huấn luyện trên tập train $\to$ tối ưu ngưỡng chi phí trên tập validation $\to$ đóng băng mô hình và ngưỡng, chỉ đánh giá duy nhất một lần trên tập test để báo cáo khách quan.